Bases de données – Clés et Modélisation Conceptuelle de Données

Outil pour l'interprétation du code SQL

Vous êtes libres d'utiliser l'outil de votre choix pour exécuter vos requêtes SQL, car l'objectif de ce module est pédagogique. On vous propose l'outil OneCompile qui peut être utilisé en ligne facilement pour vous permettre d'exécuter en toutes circonstances des requêtes de tests.

L'utilisation de cet outil implique de n'exploiter que des données parfaitement fictives ou publiques, qui n'ont donc pas de caractère personnel ou sensible. Dans un contexte pédagogique, ces caractéristiques sont habituellement faciles à réunir.

Vous trouverez cet outil à l'adresse `https://onecompiler.com/postgresql`

Exercice : gestion des ventes d'une pâtisserie

Une pâtisserie souhaite informatiser la gestion de ses ventes. La pâtisserie vend plusieurs produits (éclairs, tartes, macarons, etc.). Pour chaque produit, on souhaite connaître :

— son numéro,

— son nom,

— son prix de vente.

Les clients peuvent passer plusieurs commandes. Pour chaque client, on souhaite enregistrer :

— son numéro,

— son nom,

— son téléphone.

Une commande est passée à une date précise par un seul client. Une commande peut contenir plusieurs produits. Un même produit peut apparaître dans plusieurs commandes. Pour chaque produit commandé, on souhaite mémoriser la quantité achetée.

1. Donnez un modèle conceptuel puis logique de la pâtisserie.
2. Avec l'outil de votre choix, donnez le script qui crée la base de données.
3. Afin de montrer le résultat de votre conception, vous allez donner les requêtes SQL qui répondent à la liste de demandes suivante :
   (a) Quels sont les clients de la pâtisserie ?
   (b) Quels sont les produits dont le prix est supérieur à 5 euro ?
   (c) Quels sont les clients dont le nom commence par la lettre A ?
   (d) Donnez les produits triés par prix croissant.
   (e) Quels sont les clients ayant passé plus d'une commande ?

(f) Quelles sont les commandes dont le montant dépasse 50 euro ?

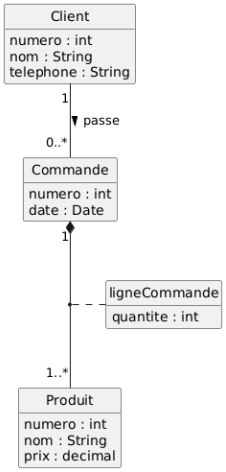
(g) Quel client a dépensé le plus ?

(h) Quelle commande contient le plus grand nombre d'articles ?

(i) Quels sont les produits qui n'ont jamais été commandés ?

Correction

Modèle conceptuel (MCD)



Modèle logique (MLD)

PRODUIT(numero, nom, prix)
CLIENT(numero, nom, telephone)
COMMANDE(numero, date, #numero_client)
LIGNE_COMMANDE(#numero_commande, #numero_produit, quantite)

Question 2 script de cr :ation

CREATE TABLE Produit (id_produit INT PRIMARY KEY, nom VARCHAR(50), prix DECIMAL(6,2))
CREATE TABLE Commande (id_commande INT PRIMARY KEY, date_commande DATE, id_client INT, FOREIGN KEY (id_client) REFERENCES Client(id_client))
CREATE TABLE Ligne_Commande (id_commande INT, id_produit INT, quantite INT,
PRIMARY KEY (id_commande, id_produit),
FOREIGN KEY (id_commande) REFERENCES Commande(id_commande),
FOREIGN KEY (id_produit) REFERENCES Produit(id_produit)
) ; ; ;

Question 3 requ

:tes – 2. Produits dont le prix depasse 5 euros SELECT * FROM PRODUIT WHERE prix > 5

– 3. Clients dont le nom commence par 'A' SELECT * FROM CLIENT WHERE nom LIKE 'A

– 4. Produits tries par prix croissant SELECT * FROM PRODUIT ORDER BY prix ASC

– 5. Clients ayant passe plus d'une commande SELECT DISTINCT numero_client FROM COMMANDE

– Mais pas le nom du client, donc limit ; ; ;

SELECT DISTINCT c.id_client, c.nom FROM CLIENT c INNER JOIN COMMANDE m ON m.id_client = c.id_client

– 6. Commandes dont le montant depasse 50 euros SELECT d.numero_commande, SUM(d.quantite * p.prix) AS montant FROM LIGNe_COMMANDE d JOIN PRODUIT p ON p.numero = d.numero_produit GROUP BY d.numero_commande HAVING SUM(d.quantite * p.prix) > 50

– 7. Client ayant depense le plus SELECT c.nom, SUM(d.quantite * p.prix) AS total FROM CLIENT c JOIN COMMANDE m ON m.numero_client = c.numero JOIN Ligne_commande d ON d.numero_commande = m.numero JOIN PRODUIT p ON p.numero = d.numero_produit GROUP BY c.nom ORDER BY total DESC LIMIT 1

– 8. Commande contenant le plus grand nombre d'articles SELECT numero_commande, SUM(quantite) AS nn_articles FROM Ligne_commande GROUP BY numero_commande ORDER BY nn_articles DESC LIMIT 1

– 9. Produits jamais commandes (anti-jointure) SELECT p.nom FROM PRODUIT p LEFT JOIN Ligne_commande d ON d.numero_produit = p.numero WHERE d.numero_produit IS NULL ; ; ; ; ;

Réalisation de la Base de données répondant au besoin d'un collège

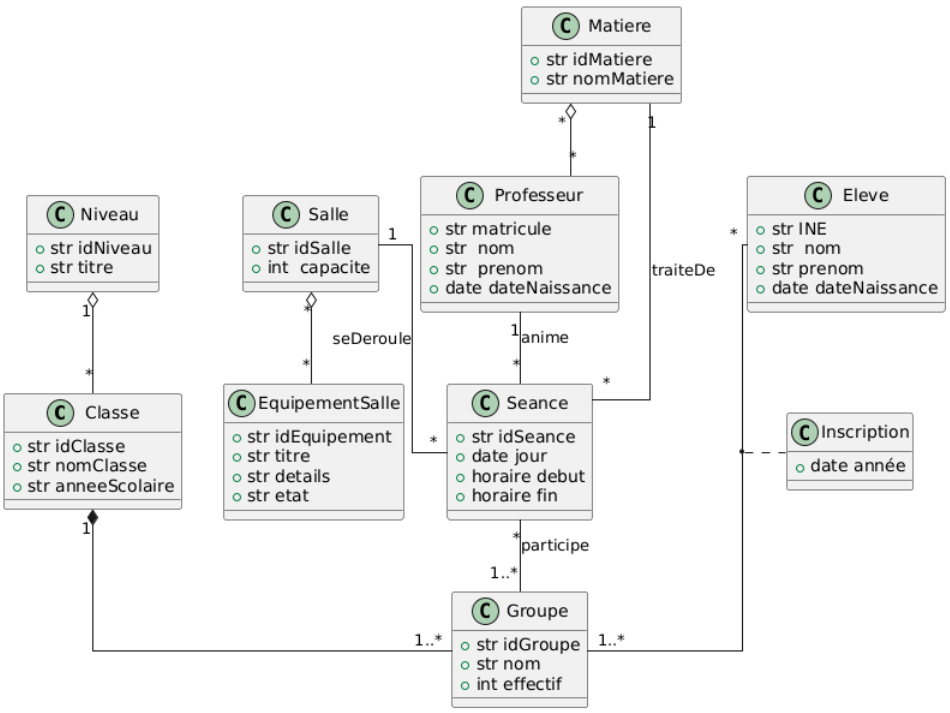
Un collège souhait administrer son fonctionnement via une base de données. Il gère des élèves, qui ont chacun un identifiant d'élève (INE), un nom, un prénom, et un niveau, correspondant à la classe dans laquelle ils sont. Chaque classe est composée de plusieurs groupes (utile pour les séances en demi-groupes, le cas de différentes langues vivantes ou options).

Il se sert de la base de données pour affecter les séances de cours dans des salles dynamiquement, en fonction de l'emploi du temps des élèves comme des professeurs et des effectifs des groupes et des ressources de chaque salle.

Réalisez en UML le MCD correspondant à la situation du collège, puis transformez-le en MLD.

À partir du MLD obtenu, proposez, pour chaque table, l'instruction SQL qui permettrait de la créer.

Correction



- NIVEAU(idNiveau, titre)
- CLASSE(idClasse, nomClasse, anneeScolaire, #idNiveau)
- SALLE(idSalle, capacite)
- EQUIPEMENT_SALLE(idEquipement, titre, details, etat)
- MATIERE(idMatiere, nomMatiere)
- PROFESSEUR(matricule, nom, prenom, dateNaissance)
- SEANCE(idSeance, jour, debut, fin, #idSalle, #matricule, #idMatiere)
- GROUPE(idGroupe, nom, effectif, #idClasse)
- ELEVE(INE, nom, prenom, dateNaissance, #idClasse)
- INSCRIPTION(#INE, #idGroupe, annee)
- EQUIPER(#idSalle, #idEquipement)
- ENSEIGNER(#matricule, #idMatiere)
- PARTICIPE(#idSeance, #idGroupe)

```
CREATE TABLE SALLE ( idSalle VARCHAR(10) NOT NULL,
capacite INT NOT NULL, PRIMARY KEY (idSalle) )
```

```
CREATE TABLE EQUIPEMENT ( idEquipement VARCHAR(20) NOT
NULL, titre VARCHAR(50) NOT NULL, details VARCHAR(255),
etat VARCHAR(20) NOT NULL, PRIMARY KEY (idEquipement) )
CREATE TABLE MATIERE ( idMatiere VARCHAR(10) NOT NULL,
nomMatiere VARCHAR(50) NOT NULL, PRIMARY KEY (idMatiere) )
CREATE TABLE PROFESSEUR ( matricule VARCHAR(20) NOT NULL,
nom VARCHAR(50) NOT NULL, prenom VARCHAR(50) NOT NULL,
dateNaissance DATE, PRIMARY KEY (matricule) )
CREATE TABLE CLASSE ( idClasse VARCHAR(10) NOT NULL,
nomClasse VARCHAR(20) NOT NULL, anneeScolaire VARCHAR(9)
NOT NULL, idNiveau VARCHAR(10) NOT NULL, PRIMARY KEY
(idClasse), FOREIGN KEY (idNiveau) REFERENCES
NIVEAU(idNiveau) )
CREATE TABLE GROUPE ( idGroupe VARCHAR(10) NOT NULL, nom
VARCHAR(50) NOT NULL, effectif INT NOT NULL, idClasse
VARCHAR(10) NOT NULL, PRIMARY KEY (idGroupe), FOREIGN KEY
(idClasse) REFERENCES CLASSE(idClasse) )
CREATE TABLE ELEVE ( INE VARCHAR(20) NOT NULL, nom
VARCHAR(50) NOT NULL, prenom VARCHAR(50) NOT NULL,
dateNaissance DATE, idClasse VARCHAR(10) NOT NULL, PRIMARY
KEY (INE), FOREIGN KEY (idClasse) REFERENCES
CLASSE(idClasse) )
CREATE TABLE INSCRIPTION ( INE VARCHAR(20) NOT NULL,
idGroupe VARCHAR(10) NOT NULL, annee VARCHAR(9) NOT NULL,
PRIMARY KEY (INE, idGroupe, annee), FOREIGN KEY (INE)
REFERENCES ELEVE(INE), FOREIGN KEY (idGroupe) REFERENCES
GROUPE(idGroupe) )
CREATE TABLE EQUIPER ( idSalle VARCHAR(10) NOT NULL,
idEquipement VARCHAR(20) NOT NULL, PRIMARY KEY (idSalle,
idEquipement), FOREIGN KEY (idSalle) REFERENCES
SALLE(idSalle), FOREIGN KEY (idEquipement) REFERENCES
EQUIPEMENT_SALLE(idEquipement))
CREATE TABLE ENSEIGNER ( matricule VARCHAR(20) NOT NULL,
idMatiere VARCHAR(10) NOT NULL, PRIMARY KEY (matricule,
idMatiere), FOREIGN KEY (matricule) REFERENCES
PROFESSEUR(matricule), FOREIGN KEY (idMatiere) REFERENCES
MATIERE(idMatiere) )
CREATE TABLE SEANCE ( idSeance INT NOT NULL, jour DATE NOT
NULL, debut TIME NOT NULL, fin TIME NOT NULL, idSalle
VARCHAR(10) NOT NULL, matricule VARCHAR(20) NOT NULL,
idMatiere VARCHAR(10) NOT NULL, PRIMARY KEY (idSeance),
FOREIGN KEY (idSalle) REFERENCES SALLE(idSalle), FOREIGN
KEY (matricule) REFERENCES PROFESSEUR(matricule), FOREIGN
KEY (idMatiere) REFERENCES MATIERE(idMatiere) )
CREATE TABLE PARTICIPE ( idSeance INT NOT NULL, idGroupe
VARCHAR(10) NOT NULL, PRIMARY KEY (idSeance, idGroupe),
FOREIGN KEY (idSeance) REFERENCES SEANCE(idSeance),
FOREIGN KEY (idGroupe) REFERENCES GROUPE(idGroupe))
;;;;;;;;;;;;
```

Analyse d'un fichier de données

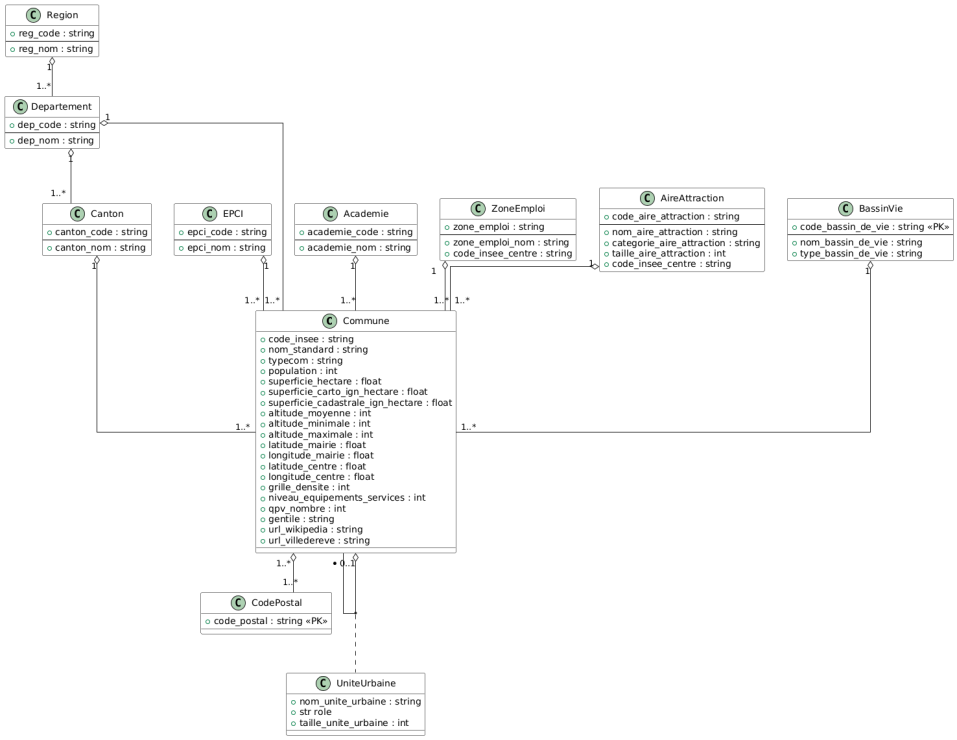
Sur le site de l'INSEE, il est possible de télécharger (`https://www.data.gouv.fr/api/1/datasets/r/c63fd0b1-7987-46f6-b779-8b3ed889090c`) un fichier contenant la liste des communes de France. À partir de la lecture de ce fichier, proposez un MCD en UML qui permettrait à l'État de gérer les collectivités territoriales en évitant les redondances pouvant mener à des pertes d'intégrité.

Transformez ensuite ce MCD en MLD décrivant les tables nécessaires, puis proposez, en SQL, les requêtes de création de ces tables.

Utilisez le fichier de données pour remplir chacune de vos tables avec au moins 5 entrées.

Correction

Globalement, il faut identifier les attributs qui se répètent, et on aboutit globalement à ça :



Il y a quelques marges d'interprétation, comme les notions d'aire urbaine, zone d'emplois, bassin de vie et d'aire d'attraction, qui peuvent être des classes en elles-mêmes ou être des relations réflexives sur commune (car généralement, l'aire d'attraction d'une commune est centrée la commune principale de cette aire d'attraction).

Faire attention aux codes postaux, une commune peut en avoir plusieurs, un code postal peut concerner plusieurs communes, car cela dépend de la poste, et les exemples sont nombreux dans les deux sens, ce n'est pas du tout anecdotique.

— `Region`(reg_code, reg_nom)
— `Departement`(dep_code, dep_nom, #reg_code)
— `Canton`(canton_code, canton_nom, #dep_code)

— `EPCI`(epci_code, epci_nom)
— `Academie`(academie_code, academie_nom)
— `ZoneEmploi`(zone_emploi, zone_emploi_nom, #code_insee_centre)
— `UniteUrbaine`(code_unite_urbaine, nom_unite_urbaine, taille_unite_urbaine)
— `AireAttraction`(code_aire_attraction, nom_aire_attraction, categorie_aire_attraction, taille_aire_attraction, #code_insee_centre)
— `BassinVie`(code_bassin_de_vie, nom_bassin_de_vie, type_bassin_de_vie)
— `CodePostal`(code_postal)
— `Commune`(code_insee, nom_standard, typecom, population, superficie_hectare, superficie_carto_ign_hectare, superficie_cadastrale_ign_hectare, altitude_moyenne, altitude_minimale, altitude_maximale, latitude_mairie, longitude_mairie, latitude_centre, longitude_centre, grille_densite, niveau_equipements_services, qpv_nombre, gentile, url_wikipedia, url_villedereve, #dep_code, #canton_code, #epci_code, #academie_code, #zone_emploi, #code_unite_urbaine, type_commune_unite_urbaine, statut_commune_unite_urbaine, #code_aire_attraction, #code_bassin_de_vie)
— `CommuneCodePostal`(#code_insee, #code_postal)

et pour le SQL :

```
CREATE TABLE Region ( reg_code CHAR(2) PRIMARY KEY,
    reg_nom VARCHAR(100) NOT NULL )
CREATE TABLE Departement ( dep_code CHAR(2) PRIMARY KEY,
    dep_nom VARCHAR(100) NOT NULL, reg_code CHAR(2) NOT
    NULL REFERENCES Region(reg_code) )
CREATE TABLE Canton ( canton_code VARCHAR(4) PRIMARY KEY,
    canton_nom VARCHAR(100) NOT NULL, dep_code CHAR(2) NOT
    NULL REFERENCES Departement(dep_code) )
CREATE TABLE EPCI ( epci_code CHAR(9) PRIMARY KEY,
    epci_nom VARCHAR(150) NOT NULL )
CREATE TABLE Academie ( academie_code CHAR(2) PRIMARY KEY,
    academie_nom VARCHAR(100) NOT NULL )
CREATE TABLE ZoneEmploi ( zone_emploi CHAR(4) PRIMARY KEY,
    zone_emploi_nom VARCHAR(150) NOT NULL,
    code_insee_centre CHAR(5) NOT NULL - FK ajout
;;;;;e aprs Commune )
CREATE TABLE UniteUrbaine ( code_unite_urbaine CHAR(5)
    PRIMARY KEY, nom_unite_urbaine VARCHAR(150) NOT NULL,
    taille_unite_urbaine INT NOT NULL CHECK
    (taille_unite_urbaine >= 0) )
CREATE TABLE AireAttraction ( code_aire_attraction CHAR(5)
    PRIMARY KEY, nom_aire_attraction VARCHAR(150) NOT
    NULL, categorie_aire_attraction VARCHAR(50),
    taille_aire_attraction INT CHECK
    (taille_aire_attraction >= 0), code_insee_centre
    CHAR(5) NOT NULL - FK ajout;;e aprs Commune )
CREATE TABLE BassinVie ( code_bassin_de_vie CHAR(5)
    PRIMARY KEY, nom_bassin_de_vie VARCHAR(150) NOT NULL,
    type_bassin_de_vie VARCHAR(50) )
-
    ================================================================
    = CODE_POSTAL =================================================
```

```
CREATE TABLE CodePostal ( code_postal CHAR(5) PRIMARY KEY
    CHECK (code_postal  '[0 - 9]5') )
-
    ============================================================
    - ENTIT;;~; PRINCIPALE     COMMUNE -
    ============================================================
CREATE TABLE Commune ( code_insee CHAR(5) PRIMARY KEY,
    nom_standard VARCHAR(150) NOT NULL, typecom VARCHAR(5)
    NOT NULL DEFAULT 'COM', - D:mographie / superficie
    population INT CHECK (population >= 0),
    superficie_hectare FLOAT CHECK (superficie_hectare >
    0), superficie_carto_ign_hectare FLOAT,
    superficie_cadastrale_ign_hectare FLOAT, - Altitudes
    altitude_moyenne INT, altitude_minimale INT,
    altitude_maximale INT, - Coordonnes latitude_mairie
    FLOAT CHECK (latitude_mairie BETWEEN 41 AND 52),
    longitude_mairie FLOAT CHECK (longitude_mairie BETWEEN
    -5 AND 10), latitude_centre FLOAT CHECK
    (latitude_centre BETWEEN 41 AND 52), longitude_centre
    FLOAT CHECK (longitude_centre BETWEEN -5 AND 10), -
    Classification grille_densite INT CHECK
    (grille_densite BETWEEN 1 AND 9),
    niveau_equipements_services INT CHECK
    (niveau_equipements_services BETWEEN 0 AND 3),
    qpv_nombre INT CHECK (qpv_nombre >= 0), - Gentil
    gentile VARCHAR(150), - Liens externes url_wikipedia
    VARCHAR(300), url_villedereve VARCHAR(300), -
    Rattachements administratifs (cls trangres) dep_code
    CHAR(2) NOT NULL REFERENCES Departement(dep_code),
    canton_code VARCHAR(4) REFERENCES Canton(canton_code),
    epci_code CHAR(9) REFERENCES EPCI(epci_code),
    academie_code CHAR(2) REFERENCES
    Academie(academie_code), zone_emploi CHAR(4)
    REFERENCES ZoneEmploi(zone_emploi), code_unite_urbaine
    CHAR(5) REFERENCES UniteUrbaine(code_unite_urbaine),
    type_commune_unite_urbaine VARCHAR(1) CHECK
    (type_commune_unite_urbaine IN ('C','H')),
    statut_commune_unite_urbaine VARCHAR(1) CHECK
    (statut_commune_unite_urbaine IN ('0','1','2')),
    code_aire_attraction CHAR(5) REFERENCES
    AireAttraction(code_aire_attraction),
    code_bassin_de_vie CHAR(5) REFERENCES
    BassinVie(code_bassin_de_vie),
- Contraintes de cohrence interne CONSTRAINT
    chk_altitude_min_le_moy CHECK (altitude_minimale <=
    altitude_moyenne), CONSTRAINT chk_altitude_moy_le_max
    CHECK (altitude_moyenne <= altitude_maximale),
    CONSTRAINT chk_typecom CHECK (typecom IN
    ('COM','ARR')), CONSTRAINT chk_altitude_consiste CHECK
    (altitude_minimale IS NULL OR altitude_maximale IS
    NULL OR altitude_minimale <= altitude_maximale)
    )
-
    ============================================================
    - TABLE DE JOINTURE  COMMUNE ;: CODE POSTAL (n    n) -
    ============================================================
CREATE TABLE CommuneCodePostal ( code_insee CHAR(5) NOT
    NULL REFERENCES Commune(code_insee), code_postal
    CHAR(5) NOT NULL REFERENCES CodePostal(code_postal),
    PRIMARY KEY (code_insee, code_postal) )
-
    ============================================================
    - FK R:;TROACTIVES ( dclarer aprs Commune) -
    ============================================================
ALTER TABLE ZoneEmploi ADD CONSTRAINT
    fk_zone_emploi_centre FOREIGN KEY (code_insee_centre)
    REFERENCES Commune(code_insee)
ALTER TABLE AireAttraction ADD CONSTRAINT
```

```
    fk_aire_attraction_centre FOREIGN KEY
    (code_insee_centre) REFERENCES Commune(code_insee) ;;
```

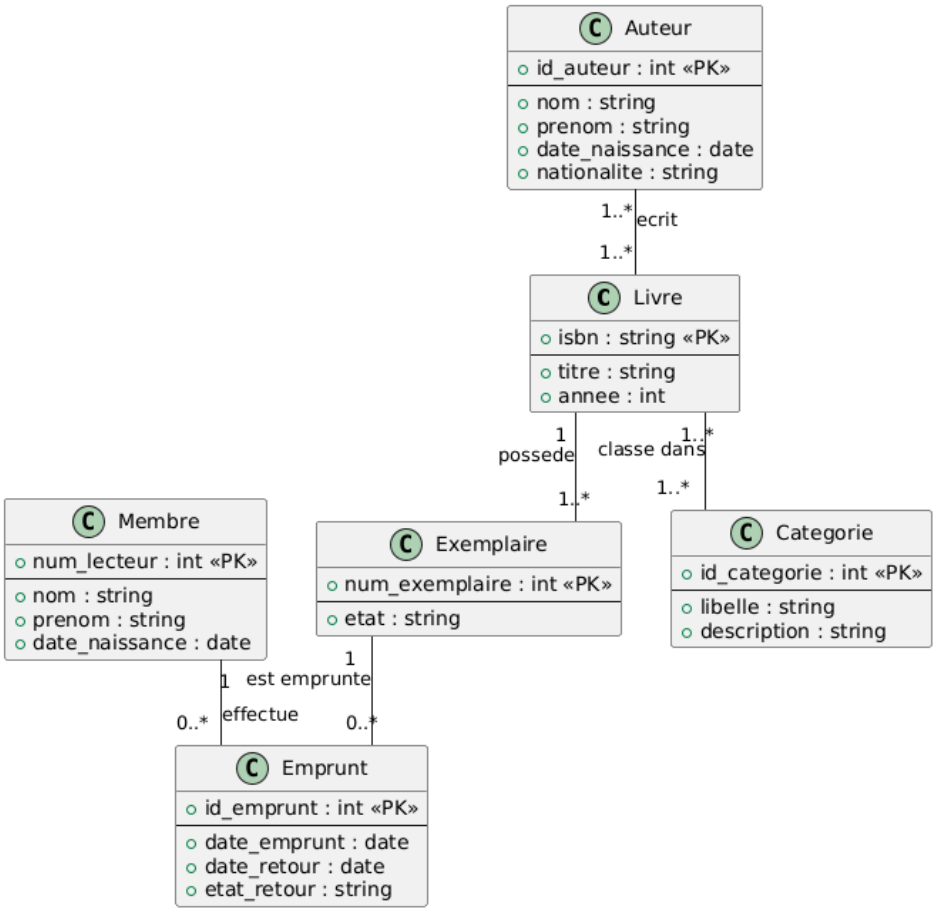
Bibliothèque

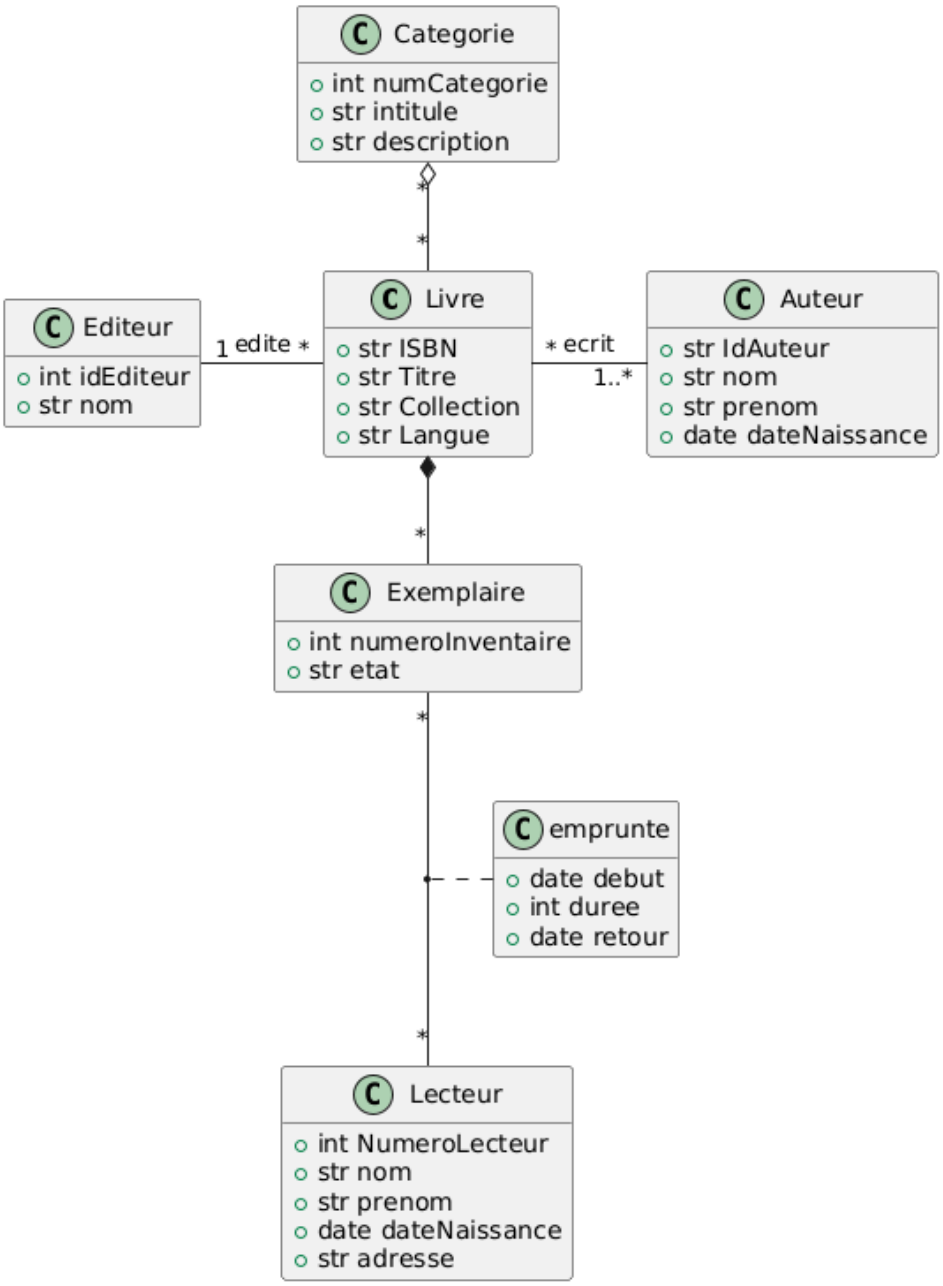
Une bibliothèque souhaite gérer ses opérations courantes liées aux emprunts sous la forme d'une Base de Données. Elle y stockera des livres, ayant un ISBN, un ou plusieurs auteurs (caractérisés par un nom, un prénom, une date de naissance, une nationalité). Elle possède potentiellement plusieurs exemplaires de chaque livre, dont elle souhaite conserver l'état courant afin de vérifier sa conformité à chaque retour.

Les membres ont un numéro de lecteur, un nom, un prénom et une date de naissance. La bibliothèque souhaite conserver une trace de l'historique de chaque emprunteur et pouvoir vérifier à chaque retour l'état du livre retourné. La bibliothèque classe les livres par catégories, pour faciliter les recherches et les recommandations. Chaque livre peut faire partie de plusieurs catégories, qui disposent chacune d'un libellé et d'une description.

Proposez un MCD correspondant au besoin de la bibliothèque, transformez-le en MLD, puis réalisez les requêtes SQL permettant de créer les tables.

Correction


Auteur
id_auteur : int «PK»
nom : string
prenom : string
date_naissance : date
nationalite : string
1..*
ecrit
1..*
Livre
isbn : string «PK»
titre : string
annee : int
1
possede
1..*
1..*
classe dans
1..*
Membre
num_lecteur : int «PK»
nom : string
prenom : string
date_naissance : date
Exemplaire
num_exemplaire : int «PK»
etat : string
Categorie
id_categorie : int «PK»
libelle : string
description : string
1
est emprunte
1
effectue
0..*
0..*
Emprunt
id_emprunt : int «PK»
date_emprunt : date
date_retour : date
etat_retour : string



— `Auteur`(id_auteur, nom, prenom, date_naissance, nationalite)
— `Livre`(isbn, titre, annee)
— `LivreAuteur`(#id_auteur, #isbn)
— `Exemplaire`(num_exemplaire, #isbn, etat)
— `Membre`(num_lecteur, nom, prenom, date_naissance)
— `Categorie`(id_categorie, libelle, description)

— `LivreCategorie`(#isbn, #id_categorie)
— `Emprunt`(id_emprunt, #num_lecteur, #num_exemplaire, date_emprunt, date_retour, etat_retour)